NumPy Arrays in One Dimension

Array creation and dtypes, universal functions, reductions, basic indexing and views, advanced indexing by integer array and by boolean array, assignment through an index, and locating matches. All examples are one-dimensional.

01 Arrays and lists
02 Array creation and attributes
03 Universal functions
04 Reductions
05 Basic indexing and views
06 Advanced indexing
07 Assignment through an index
08 Positions of matching elements
09 Loop forms and array equivalents

Scope. A Python list stores references to objects scattered across memory, and every operation on it runs in the interpreter one element at a time. A NumPy array stores raw values of a single type in one contiguous block, and operations on it run in compiled code that does not return to the interpreter between elements. Consequences of that difference include the speed of array operations, the fixed element type, and the fact that a slice does not copy.

After reading it you should be able to do the following without looking anything up.

- Create arrays with `np.array`, `np.zeros`, `np.ones`, `np.arange` and `np.linspace`, and read `shape`, `dtype`, `ndim` and `size`.
- Apply arithmetic and comparison to a whole array without writing a loop.
- Reduce an array to a single value with `sum`, `mean`, `min` and `max`.
- Select elements by basic indexing, by integer array and by boolean array, and say which of those returns a view and which returns a copy.
- Write into an array through any of those three, and find the first element that satisfies a condition.

Every example assumes `import numpy as np`.

01

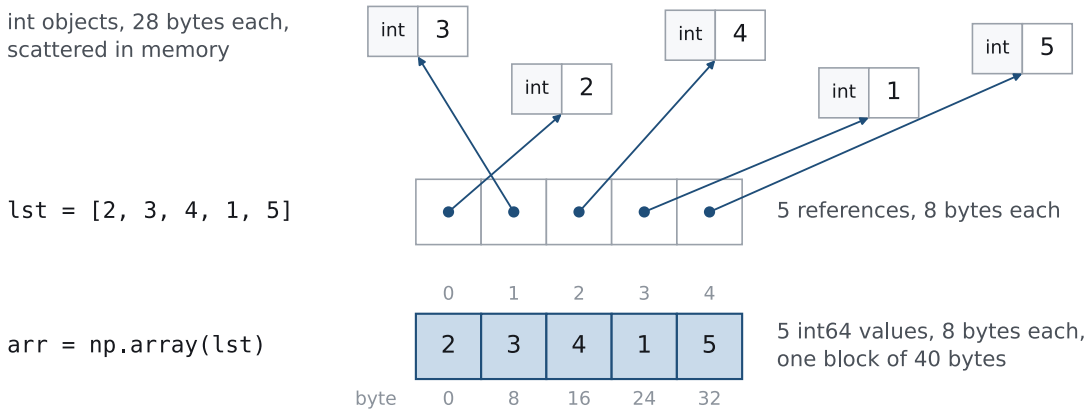
Arrays and lists

A list and an array holding the same million integers differ in memory layout, total size and per-element access cost.

MEMORY LAYOUT

A list owns a contiguous block of references. The integers themselves are separate Python objects elsewhere in memory, each with its own header. On this machine a small `int` costs 28 bytes, so a list of a million of them is about 8 MB of references plus roughly 28 MB of objects.

An array owns a contiguous block of values. A million `int64` values are 8 bytes each and nothing else: 8 MB in total, packed end to end, with no per-element object.



The list [2, 3, 4, 1, 5] and the array built from it. The list stores 8-byte references to separate int objects of 28 bytes each; the int64 array stores the values themselves, 8 bytes each, at consecutive byte offsets.

That has three consequences. The data occupies about a quarter of the memory. Reading an element is one memory access rather than two. And because the values are adjacent, the processor's cache and its vector instructions can operate on many of them at a time.

It has two costs.

- **The type is fixed at creation.** Every element occupies the same number of bytes and is interpreted the same way, so an array cannot hold an integer next to a string the way a list can. That type is the array's `dtype`.
- **The size is fixed at creation.** `np.append` returns a new array rather than growing the existing one. Growing an array means allocating a new one and copying, so arrays are built once at their final size rather than accumulated element by element.

MEASURED COST

Summing the squares of n numbers, as a Python loop over a list and as an array expression. Best of nine runs, on Apple silicon with Python 3.14 and NumPy 2.5:

N	SUM(X * X FOR X IN LST)	(ARR * ARR).SUM()	RATIO
100 000	0.0031 s	0.000048 s	65×
1 000 000	0.0340 s	0.00082 s	41×
10 000 000	0.4050 s	0.0106 s	38×

Both versions are `O(n)`: each row's times are roughly ten times the row above it. NumPy removes the per-element interpreter overhead, which is a large constant factor.

The ratio is not constant: it falls from 65× to about 38× as the data stops fitting in cache and both versions become more limited by memory access than by arithmetic. Exact figures depend on the machine, the Python version and the NumPy build, so only the order of magnitude is comparable across machines.

NumPy therefore does not change the complexity of an algorithm: a quadratic algorithm stays quadratic, and a linear pass becomes cheaper by a large constant factor.

Replacing a Python loop with an array-to-array operation, so that the looping happens in compiled code rather than in the interpreter, is called **vectorization**.

02

Array creation and attributes

Arrays are constructed from existing data, or from a shape and a fill value. Four attributes describe the result: `shape`, `dtype`, `ndim` and `size`.

```
a = np.array([2, 3, 4, 1, 5], dtype=np.int32)   # (5,)

a.shape        # -> (5,)             length along each axis, as a tuple
a.dtype        # -> dtype('int32')   the type of every element
a.ndim         # -> 1                number of axes, same as len(a.shape)
a.size         # -> 5                total elements, the product of shape
a.itemsize     # -> 4                bytes per element
len(a)         # -> 5                length along axis 0
```

The nesting depth of the input lists determines the number of axes, and every list at the same depth must be the same length. Ragged input raises an error instead of producing a jagged array:

```
np.array([[1, 2], [3]])   # row lengths 2 and 1
# ValueError: setting an array element with a sequence. The requested array
# has an inhomogeneous shape after 1 dimensions.
```

CREATION FROM A SHAPE

```
np.zeros(5)                  # -> (5,)   [0. 0. 0. 0. 0.], float64 by default
np.ones(3, dtype=np.int32)   # -> (3,)   [1 1 1]
np.full(4, 7)                # -> (4,)   [7 7 7 7]
np.zeros_like(a)             # -> (5,)   a's shape and a's dtype
np.arange(5)                 # -> (5,)   [0 1 2 3 4], stop excluded as in range
np.arange(2, 10, 3)          # -> (3,)   [2 5 8]
np.arange(10, 2, -3)         # -> (3,)   [10 7 4], a negative step walks back
np.arange(2, 10, -2)         # -> (0,)   [], start below stop, negative step
np.linspace(0, 1, 5)         # -> (5,)   [0. 0.25 0.5 0.75 1.], endpoint included
```

`np.arange` takes a step and excludes the stop, like `range`. `np.linspace` takes a count and includes both ends, which suits evenly spaced samples of an interval. Unlike `range`, `np.arange` is not lazy: it allocates every value immediately.

DTYPE

Without an explicit dtype, NumPy infers one: integer input gives `int64`, anything containing a float gives `float64`, and `np.zeros` and `np.ones` give `float64`. `astype` makes a new array with a different type, copying as it converts:

```
vals = np.array([-2, 3, -4], dtype=np.int32)   # (3,) int32

np.fabs(vals)                    # (3,) -> (3,) float64   [2. 3. 4.]
np.fabs(vals).astype(np.int32)   # (3,) -> (3,) int32     [2 3 4], a new array
```

A NumPy dtype is not the same thing as a Python type. Every Python object has a type, and the type of a NumPy array is `ndarray`. The dtype describes the elements inside it.

INTEGER OVERFLOW

A Python `int` has no fixed width. A NumPy integer has a fixed width, and exceeding it wraps rather than raising. No warning is printed, and the wrong value is the only sign of the overflow.

```
big = np.array([2_000_000_000, 2_000_000_000], dtype=np.int32)   # (2,) int32
big + big        # (2,) v (2,) -> (2,)   [-294967296 -294967296]

small = np.array([100, 200], dtype=np.uint8)                     # (2,) uint8
small * 2        # (2,) v ()   -> (2,)   [200 144], and 144 is 400 - 256
```

NumPy checks the range only when an array is built: `np.array([400], dtype=np.uint8)` raises `OverflowError`. Converting does not, so `np.array([400]).astype(np.uint8)` gives 144. Widening works only before the computation: when a squared or accumulated value might leave the range, the array is converted with `arr.astype(np.int64)` first.

03

Universal functions

A universal function, or **ufunc**, operates on an `ndarray` element by element in compiled code. The arithmetic and comparison operators are ufuncs in operator form.

Adding two arrays of the same shape, written out by hand, is a loop:

```
arr1 = np.array([1, 2, 3])    # (3,)
arr2 = np.array([10, 20, 30]) # (3,)

out = np.zeros_like(arr1)     # (3,)
for i in range(arr1.shape[0]):
    out[i] = arr1[i] + arr2[i]

out = arr1 + arr2             # (3,) v (3,) -> (3,)   the same, in compiled code
out = np.add(arr1, arr2)      # + is the operator form of np.add
```

Every operator is a ufunc with a name, and the two spellings are interchangeable: `-` is `np.subtract`, `*` is `np.multiply`, `/` is `np.true_divide`, `//` is `np.floor_divide`, `%` is `np.mod`, `**` is `np.power`. As in plain Python, `/` always produces floats and `//` floors towards negative infinity.

The term appears in the documentation and in error messages. A ufunc supports broadcasting, type casting and keyword arguments such as `out=`, and one reference page documents all of these.

Comparisons work the same way and produce a boolean array rather than a single `True`:

```
arr = np.array([2, 3, 4, 1])  # (4,)

arr > 2                       # (4,) -> (4,) bool   [False True True False]
arr % 2 == 0                  # (4,) -> (4,) bool   [True False True False]
(arr > 2).sum()               # (4,) -> ()          2, True counts as 1
```

SCALAR OPERANDS

`arr + 42` adds 42 to every element. The scalar is treated as though it had been repeated to the array's shape, without actually being copied. This is the simplest case of broadcasting. Two 1D arrays of different lengths do not combine:

```
np.array([1, 2]) + np.array([1, 2, 3])   # (2,) v (3,)
# ValueError: operands could not be broadcast together with shapes (2,) (3,)
```

NP.MAXIMUM AGAINST NP.AMAX

Two functions with similar names and different behaviour:

```
x = np.array([2, 3, 4, 1, 5])   # (5,)

np.maximum(x, 3)     # (5,) v () -> (5,)   [3 3 4 3 5], element-wise
np.amax(x)           # (5,)      -> ()     5, a reduction
x.max()              # (5,)      -> ()     5, the method form of np.amax
```

`np.maximum` and `np.minimum` are ufuncs: they compare two arrays element by element and return an array of the same shape. `np.amax` and `np.amin` are reductions: they reduce the array to a single value. `np.maximum(arr, 0)` is the standard way to clamp negatives to zero.

THE OUT= KEYWORD

Every ufunc allocates a new array for its result. When the array is large and the old values are not needed, `+=` or the ufunc's `out=` keyword argument write into existing memory instead:

```
# arr: (4,)

arr += 2                      # (4,) -> (4,)   in place, no new array
np.add(arr, 2, out=arr)       # the same thing, written out
np.maximum(arr, 0, out=arr)   # out= works for functions with no operator form
```

04

Reductions

A reduction applies a ufunc along one axis and reduces the array's dimensionality by one. A one-dimensional array has one axis, so the result is a single value.

```
m = np.array([2, 3, 4, 1, 5, 8])   # (6,)

m.sum()          # (6,) -> ()   23
m.prod()         # (6,) -> ()   960
m.mean()         # (6,) -> ()   3.8333..., always a float
m.min()          # (6,) -> ()   1
m.max()          # (6,) -> ()   8
m.std()          # (6,) -> ()   standard deviation
m.var()          # (6,) -> ()   variance
m.argmax()       # (6,) -> ()   5, the position of the largest value
m.argmin()       # (6,) -> ()   3, the position of the smallest
```

Ties in `argmax` and `argmin` are broken by position: the index of the first occurrence is returned.

Reductions combine with ufuncs: a ufunc transforms every element, then a reduction produces a single value.

```
values = np.array([98, 104, 91, 130])   # (4,)
scores = np.array([31, 57, 50, 72, 44])  # (5,)

((values - 100) ** 2).mean()          # (4,) -> ()     mean squared deviation
(scores >= 50).sum()                  # (5,) -> ()     3, how many passed
(scores >= 50).mean()                 # (5,) -> ()     0.6, what fraction
np.cumsum(values)                     # (4,) -> (4,)   a running total
```

`np.cumsum` is an accumulation, not a reduction. It returns an array the same length as its input, where entry *i* is the total of everything up to and including *i*, so no axis is removed. It is the array form of an accumulator loop.

> **ACCUMULATOR DTYPE**
>
> `sum` on an `int32` array accumulates in `int64`, so the total of many large `int32` values does not overflow. The ufunc that runs before it has no such protection. In `(arr * arr).sum()` the multiplication happens first, at the array's own dtype, and can wrap before the sum is computed.

On an array with more than one axis, the `axis=` argument names which axis the reduction runs along.

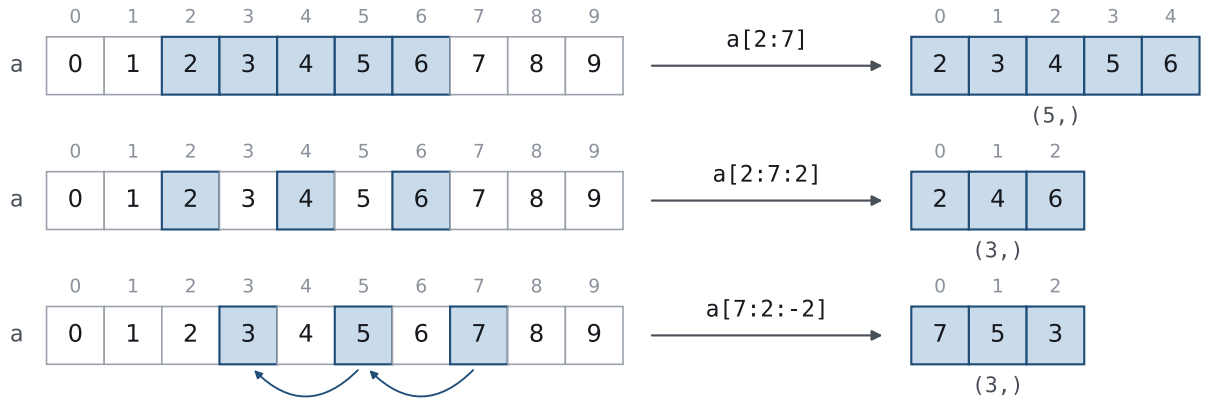
05

Basic indexing and views

Indexing with integers and slices is **basic indexing**. The syntax is identical to Python lists, but every array produced by basic slicing is a view of the original rather than a copy.

```
a = np.arange(10)             # (10,)  [0 1 2 3 4 5 6 7 8 9]

a[2:7]        # (10,) -> (5,)    [2 3 4 5 6], stop excluded
a[2:7:2]      # (10,) -> (3,)    [2 4 6], start:stop:step
a[2:]         # (10,) -> (8,)    from 2 to the end
a[:-2]        # (10,) -> (8,)    everything but the last two
a[::-1]       # (10,) -> (10,)   reversed
a[7:2:-2]     # (10,) -> (3,)    [7 5 3], negative step needs start above stop
a[2:7:-2]     # (10,) -> (0,)    [], otherwise nothing is selected
```



The elements of a = np.arange(10) selected by three slices, and the resulting arrays. The stop index is excluded; with a negative step the slice runs from start down towards stop, in the order of the curved arrows.

An out-of-range index raises an error, but an out-of-range slice is clipped:

```
# a: (10,)

a[12:]        # (10,) -> (0,)    [], no error
a[:12]        # (10,) -> (10,)   the whole array, no error
a[12]         # IndexError: index 12 is out of bounds for axis 0 with size 10
```

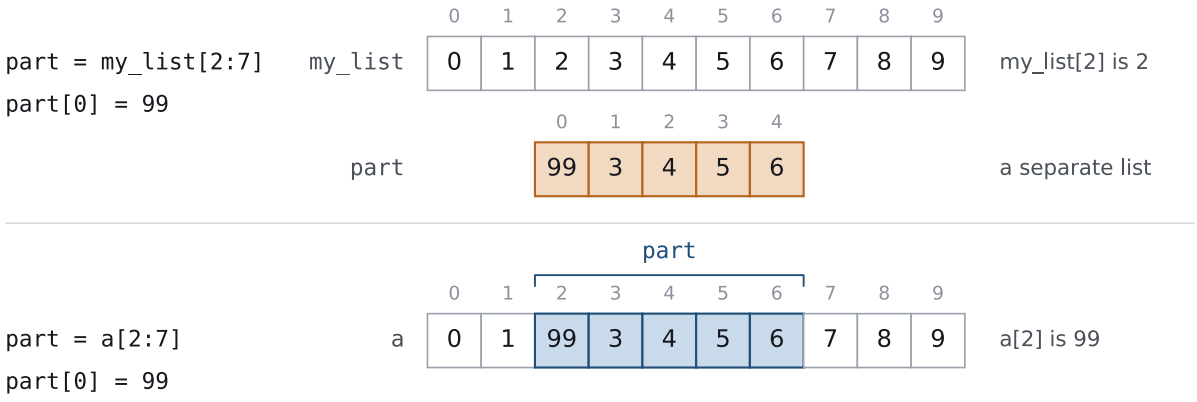
A window computed slightly wrong therefore returns a short array instead of raising, and the error shows up later as a wrong number.

VIEWS

Slicing a list copies. Slicing an array does not.

```
my_list = list(range(10))   # a list of 10 ints
part = my_list[2:7]         # a new list of 5
part[0] = 99
my_list[2]                  # -> 2    the list is untouched, slicing copied

a = np.arange(10)           # (10,)
part = a[2:7]               # (10,) -> (5,)
part[0] = 99
a[2]                        # -> 99   the array changed, slicing did not copy
```



Slicing the list copies five elements into a new list, so part[0] = 99 leaves my_list unchanged. Slicing the array gives a second name, part, for five cells of the block a describes, so part[0] = 99 writes into a.

A basic slice allocates no data. It is a second `ndarray` object pointing into the same memory, differing only in where it starts, how many elements it covers, and the stride between them. Writing through either name changes the one block of memory both of them describe.

Selecting a region of a large array therefore copies no data, so slicing inside a loop does not allocate the way slicing a list does.

EXPLICIT COPIES

An explicit copy is needed when a function modifies the array passed to it, or when a piece of an array has to keep its values after the original changes. The copy no longer shares memory with the original.

```
# a: (10,)

part = a[2:7].copy()      # (10,) -> (5,)   or np.copy(a[2:7])
part[0] = -1
a[2]                      # -> 2   unchanged
```

Copying where a view would suffice costs memory and time, and the cost is noticeable in a loop over a large array.

06

Advanced indexing

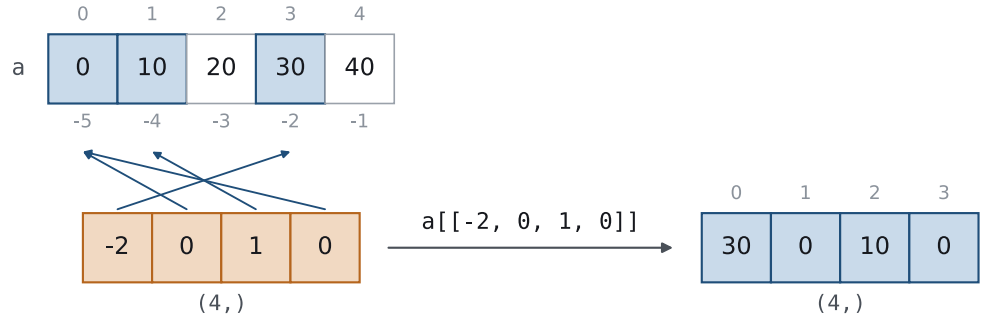
Indexing with an array rather than with integers and slices is **advanced indexing**. It comes in two forms, integer array indexing and boolean array indexing, and both always return a copy.

INTEGER ARRAY INDEXING

Indexing with a list or array of integers returns the elements at those positions, in that order. Positions may repeat and may be negative, and the result has the length of the integer array, not of the original:

```
a = np.array([0, 10, 20, 30, 40])   # (5,)

a[[2, 3]]              # (5,) -> (2,)   [20 30]
a[[-2, 0, 1, 0]]       # (5,) -> (4,)   [30 0 10 0], repeats are allowed
a[[]]                  # (5,) -> (0,)   []
a[np.array([4, 4, 4])] # (5,) -> (3,)   [40 40 40]
```



a[[-2, 0, 1, 0]]: each entry of the index array picks one element of a, counting from the end when negative. The result has the index array's length, (4,), and the element at position 0 appears twice.

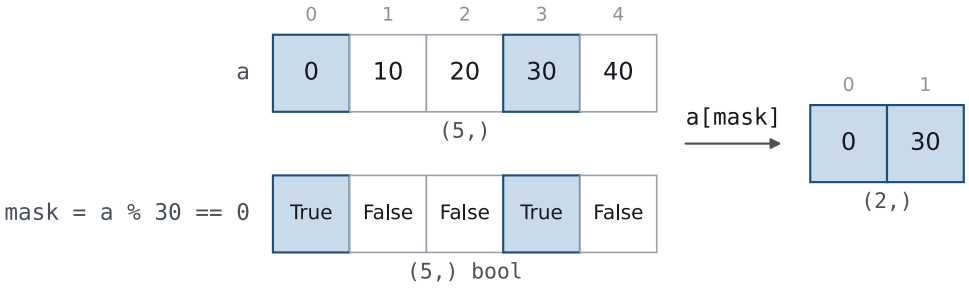
Unlike basic slicing, this returns a **copy**. The positions are arbitrary, so there is no start, length and stride that could describe them as a view.

BOOLEAN ARRAY INDEXING

A comparison produces a boolean array the same length as the original. Such an array used for selection is called a **mask**, and indexing with it keeps the elements where it is `True`:

```
a = np.array([0, 10, 20, 30, 40])   # (5,)

mask = a % 30 == 0     # (5,) -> (5,) bool   [True False False True False]
mask.dtype             # -> bool
a[mask]                # (5,) -> (2,)   [0 30]
a[a > 15]              # (5,) -> (3,)   [20 30 40], written in one step
```



mask = a % 30 == 0 has one boolean per element of a. a[mask] keeps the elements where the mask is True, in their original order, so the result has shape (2,).

A mask used for indexing must be the same length as the axis it indexes. This is the second form of advanced indexing, so it also returns a copy.

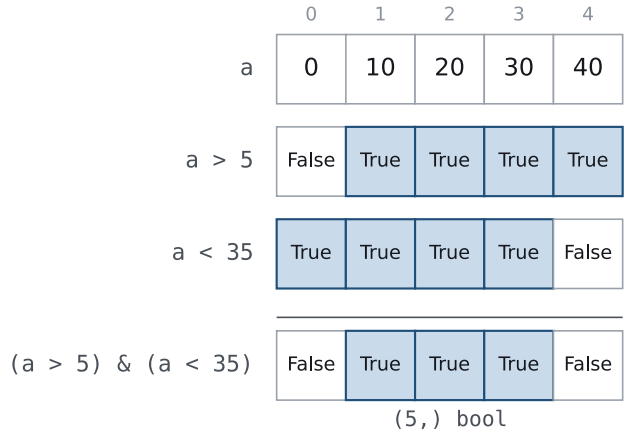
COMBINING CONDITIONS

Conditions combine with `&` for and, `|` for or, and `~` for not. Python's `and`, `or` and `not` do not work here: they require a single true-or-false answer, and an array of them raises.

```
a = np.array([0, 10, 20, 30, 40])   # (5,)

(a > 5) & (a < 35)      # (5,) v (5,) -> (5,) bool   [False True True True False]
(a < 5) | (a > 35)      # (5,) v (5,) -> (5,) bool   [True False False False True]
~(a > 15)               # (5,)        -> (5,) bool   [True True False False False]

a > 5 and a < 35
# ValueError: The truth value of an array with more than one element is
# ambiguous. Use a.any() or a.all()
```



& combines two masks of the same shape element by element: the result is True only where both masks are True.

OPERATOR PRECEDENCE

`&`, `|` and `~` bind more tightly than the comparison operators do. Without brackets around each comparison the expression is grouped the wrong way and either raises or computes a different condition without an error.

```
# a: (5,)

a > 5 & a < 35          # parsed as  a > (5 & a) < 35
(a > 5) & (a < 35)      # the intended grouping
```

COUNTING AND TESTING

Reductions on a boolean array count the `True` values, since `True` counts as 1, and test whether any or all of them are `True`:

```
# a: (5,)

mask = a > 15          # (5,) -> (5,) bool
mask.sum()             # (5,) -> ()   3, how many satisfy it
mask.mean()            # (5,) -> ()   0.6, what fraction
mask.any()             # (5,) -> ()   True, at least one is True
mask.all()             # (5,) -> ()   False, not all are True
np.count_nonzero(mask) # (5,) -> ()   3, the same count, stated explicitly
```

07

Assignment through an index

Basic slices, integer arrays and masks may all appear on the left of an assignment. The right-hand side is either a single scalar or exactly as many values as were selected.

```
a = np.array([0, 10, 20, 30, 40])   # (5,)

a[-1] = 99             # one element
a                      # -> (5,)   [ 0 10 20 30 99]

a[::2] = 42, 77, 11    # 3 positions selected, 3 values given
a                      # -> (5,)   [42 10 77 30 11]

a[::2] = 5             # 3 positions selected, one value for all of them
a                      # -> (5,)   [ 5 10  5 30  5]

a[[1, -1, 0]] = 7      # an integer array
a[a > 6] = 0           # a boolean mask
```

Supplying the wrong number of values raises an error:

```
a = np.array([0, 10, 20, 30, 40])   # (5,)
a[::2] = 42, 99                     # 3 positions selected, 2 values given
# ValueError: could not broadcast input array from shape (2,) into shape (3,)
```

CASTING ON ASSIGNMENT

The array's dtype is fixed, so an assigned value is converted to it rather than changing it. Assigning a float into an integer array truncates, without a warning.

```
a = np.array([0, 10, 20], dtype=np.int64)   # (3,) int64
a[0] = 99.99
a                      # -> [99 10 20]   not 99.99
```

Because a basic slice is a view, writing into a slice of an array is the same as writing into the array. Advanced indexing produces a copy when it is read, but assignment through it writes into the original array and makes no copy.

```
a = np.array([0, 10, 20, 30, 40])   # (5,)

picked = a[[0, 1]]     # (5,) -> (2,)   a copy
picked[0] = -1
a[0]                   # -> 0    the original is untouched

a[[0, 1]] = -1         # assignment through the same indices
a[0]                   # -> -1   the original changed
```

08

Positions of matching elements

Converting a boolean mask into positions, and the failure mode when no element satisfies the condition.

`np.nonzero` returns the positions where the mask is `True` as a tuple with one array per axis. `np.flatnonzero` returns them as one array of indices into the flattened array, which for a one-dimensional mask are the same positions:

```
a = np.array([0, 10, 20, 30, 40])   # (5,)
mask = a > 15                       # (5,) bool, 3 of them True

np.flatnonzero(mask)   # (5,) -> (3,)     [2 3 4], every matching position
np.nonzero(mask)       # (5,) -> ((3,),)  a tuple, one entry per axis
np.nonzero(mask)[0]    # -> (3,)          [2 3 4], which is why the [0] is there
```

For the first match only, `argmax` on the mask gives the position. On a boolean array it stops at the first `True` rather than scanning to the end.

```
# a: (5,)

np.argmax(a > 15)      # (5,) -> ()   2, the first position above 15
```

ARGMAX ON AN ALL-FALSE MASK

When nothing matches, every element is `False`, ties are broken by position, and `argmax` returns `0`. It does not return `-1` or raise an exception, and `0` is a valid index pointing at the first element.

```
a = np.array([0, 10, 20, 30, 40])   # (5,)

np.argmax(a > 15)      # (5,) -> ()   2, correct
np.argmax(a > 999)     # (5,) -> ()   0, and nothing is above 999 anywhere
```

The result therefore has to be checked:

```
# a: (5,)

hit = a > 999                                    # (5,) -> (5,) bool, all False
idx = int(np.argmax(hit)) if hit.any() else -1   # -> -1
```

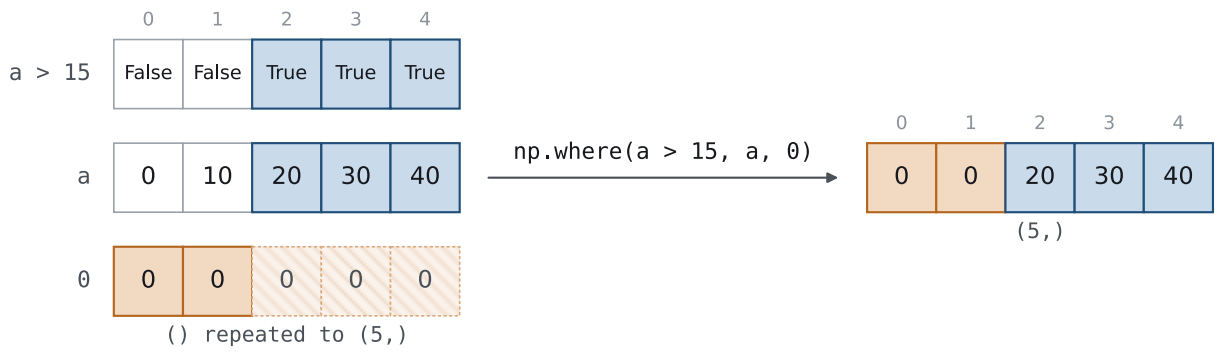
The same applies to `np.flatnonzero`, which returns an empty array rather than raising, so `np.flatnonzero(hit)[0]` raises `IndexError` on no match.

NP.WHERE

`np.where` with three arguments selects element by element from two sources according to a mask. It returns a new array and does not modify either input:

```
a = np.array([0, 10, 20, 30, 40])   # (5,)

np.where(a > 15, a, 0)        # (5,) -> (5,)   [0 0 20 30 40], keep or zero
np.where(a > 15, 1, -1)       # (5,) -> (5,)   [-1 -1 1 1 1], two scalars
```



np.where(a > 15, a, 0) takes each element from a where the condition is True and from 0 where it is False. The scalar 0 is broadcast to shape (5,); its hatched cells are the positions where it is not used.

With one argument, `np.where(mask)` is the same as `np.nonzero(mask)` and returns a tuple of positions.

09

Loop forms and array equivalents

Common loop forms and their array equivalents. Each expression in the right-hand column runs in compiled code and references no index variable.

WHAT THE LOOP DOES	AS AN ARRAY EXPRESSION
Add a constant to every element	`arr + 42`
Add two equal-length sequences	`a + b`
Running total	`np.cumsum(arr)`
Total of everything	`arr.sum()`
Count the elements passing a test	`(arr > 2).sum()`
Fraction passing a test	`(arr > 2).mean()`
Any element true, all elements true	`mask.any()`, `mask.all()`
Largest value, and where it is	`arr.max()`, `arr.argmax()`
Keep only the elements passing a test	`arr[arr > 2]`
Positions of everything passing a test	`np.flatnonzero(arr > 2)`
Position of the first element passing a test	`np.argmax(arr > 2)`, guarded
Clamp negatives to zero	`np.maximum(arr, 0)`
Replace the elements passing a test	`arr[arr > 2] = 0`
Differences between neighbours	`arr[1:] - arr[:-1]`
Total of a stretch	`arr[i0:i1].sum()`

A difference between successive elements is a subtraction of two slices offset by one, with no loop and no index arithmetic. A total over a stretch is one slice and then a reduction.

FURTHER READING

- NumPy: the absolute basics, the introduction in the NumPy documentation to array creation, attributes, indexing and slicing, and basic operations.
- Indexing on ndarrays, the reference for basic indexing and for advanced indexing by integer array and by boolean array.
- Universal functions, the reference for what the operators call, including `out=` and the type rules.
- The routine reference, every NumPy function grouped by topic.